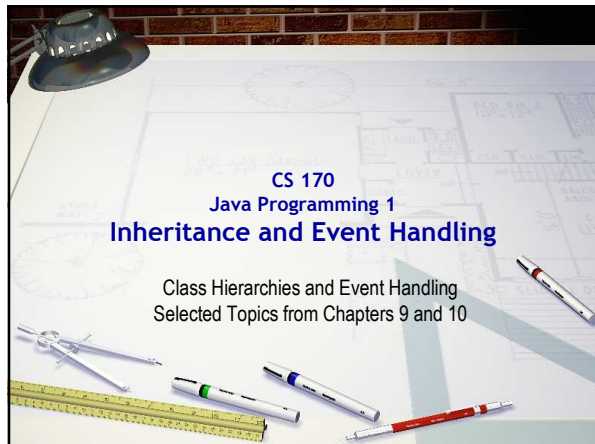
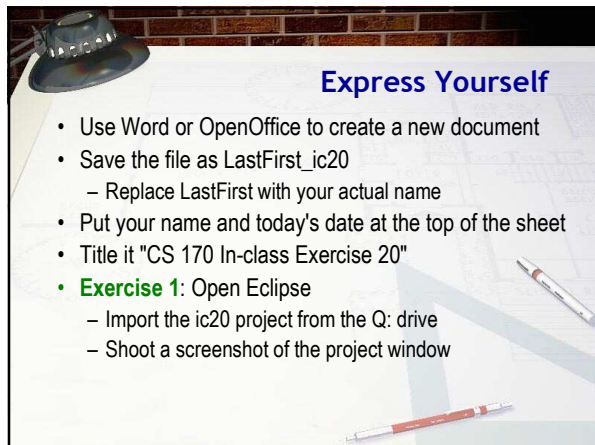
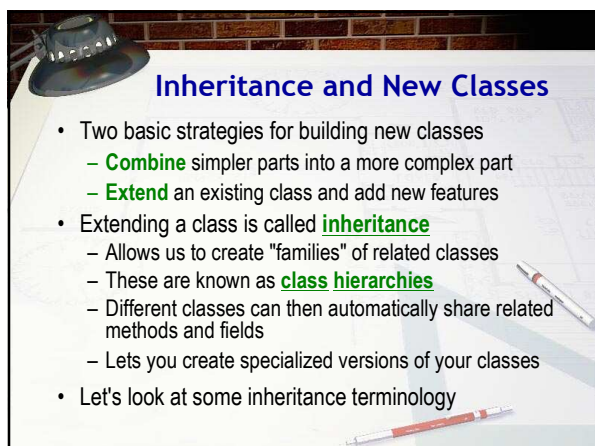








Superclass and Subclass

- Divide Animals into Insect, Mammal, Amphibian subsets
 - Subset is called a subclass
 - Specialized kind of Animal
- Combine Rodent, Primate, Elephant into Mammal
 - Mammal is a superclass
 - Represents generalization

Inheritance & Specialization

- Subclass normally represents **specialization**
 - Inherits** all attributes and methods of superclass
 - Usually adds additional methods or fields

Substitutability

- Everything a **Button** can do, an **ImageButton** can
 - Use **ImageButton** anywhere you use a **Button**
 - Called **the principle of substitutability**

```
public void methodOne(Button b)...
public void methodTwo(ImageButton b) ...
```

- Pass **ImageButton** to both, but only **Button** to first
 - Every ImageButton is-a Button**
 - Every **Button** is **not necessarily** an **ImageButton**
- Java supports only **single inheritance**

The Widget Hierarchy

- **Object** is **root** class
- **Component** is superclass for most AWT GUI objects
- **Container**: "holds" other components
- Three main branches:
 - **Panel**: superclass of **Applet**
 - **Window**: superclass of **Frame** and **JFrame**
 - **JComponent**: superclass of Swing widgets

Creating a new Widget

- Let's create a new class called **Dot**
 - Could be used as basis for any kind of "game" object
- Java has two "blank" widgets, just made for extending
 - **Canvas**: uses a "blank" native peer
 - Components are rectangular and opaque
 - **Component**: does not use a native peer
 - Can create non-rectangular and transparent regions
 - A little more difficult to use correctly

Step 1 : The Test Harness

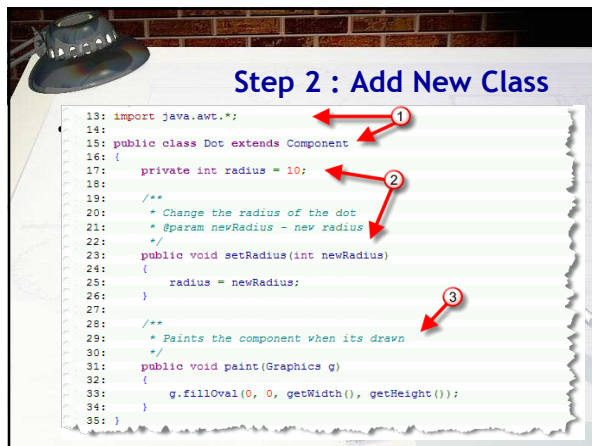
- Start by creating the applet that **tests** our new widget
- Create a new Java applet in Eclipse named **TestDots**

Step 2 : Add New Class

```

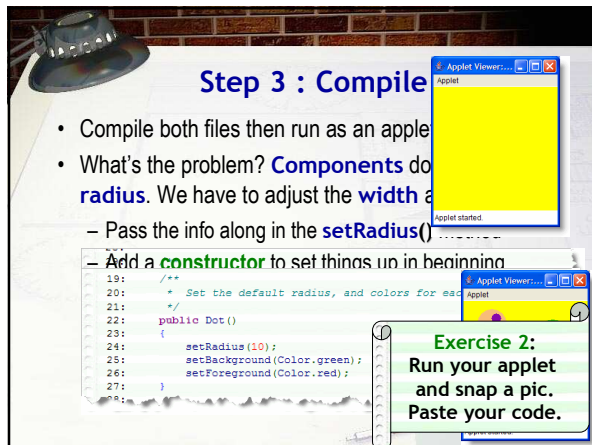
13: import java.awt.*;
14:
15: public class Dot extends Component
16: {
17:     private int radius = 10;
18:
19:     /**
20:      * Change the radius of the dot
21:      * @param newRadius - new radius
22:      */
23:     public void setRadius(int newRadius)
24:     {
25:         radius = newRadius;
26:     }
27:
28:     /**
29:      * Paints the component when its drawn
30:      */
31:     public void paint(Graphics g)
32:     {
33:         g.fillOval(0, 0, getWidth(), getHeight());
34:     }
35: }

```



Step 3 : Compile

- Compile both files then run as an applet
- What's the problem? **Components** do not have a **radius**. We have to adjust the **width** and **height** of the **Component** to match the **radius**.
 - Pass the info along in the **setRadius()**
 - Add a **constructor** to set things up in beginning



```

19: /**
20:  * Set the default radius, and colors for each Dot
21:  */
22: public Dot()
23: {
24:     setRadius(10);
25:     setBackground(Color.green);
26:     setForeground(Color.red);
27: }

```

Exercise 2:
 Run your applet and snap a pic.
 Paste your code.

Three Lessons Learned

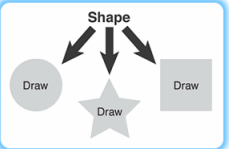
- We can create **Dot** variables and objects
 - Just as we can for **Button** or **Label** types
- The **Dot** object **is-a Component** just like those that come with the Java platform GUI libraries
 - **Container add()** expected a **Component**
- The **Dot** class **inherits all** the **Component** methods
 - Got **setBackground()**, **setSize()** for "free"
- Let's improve **Dot** by making it "mouse sensitive"
 - We need abstract classes and interfaces

Abstract Classes

- Abstract classes are "incomplete"
 - They cannot be used to **create** objects
- `new <AbstractClassName>() ;` 
 - It contains methods common to **Buttons** and **Labels**
 - You **cannot** create a **Component object**
 - You **can** create a **Component variable**
 - You **can also extend** the **Component** class
- The subclasses are called **concrete classes**

Why Abstract Classes?

- Abstract classes allow you to create **generic** classes
 - Common functionality only needs to be written once
- Know a method is needed but don't know the details
 - A program like Visio needs to **draw()** shapes
 - Each different kind of shape is drawn in a different way
 - A generic **draw()** is useless
 - Each concrete class will implement **draw()** differently
- Abstract classes will implement a **draw()** or **deal()**



Abstract Classes

- Define using the modifier **abstract** in the header

```
public abstract class Shape
{
    private int x, y;
}
```

- The **Shape** class contains fields (**x** and **y**) that can be used to store its location on the display
 - Add other fields and methods just like a regular class

Express Yourself

- **Exercise 3:** open the **Hand** class in Eclipse
 - Represents a generic hand of cards
 - Add code to make **Hand** an **abstract** class
 - Shoot a screen-shot of the code you added
- **Exercise 4:** modify **BeginnersBlackjack**
 - Add two **Hand** instance variables
 - Name one **player** and the other **dealer**
- What happens if you try to initialize the variable?
 - Shoot a screen-shot of the error

Abstract Methods

- An abstract method is a **mandatory** method
 - Use the modifier **abstract** when defining the method
 - Replace the body of the method with **a semicolon**

```
public abstract class Shape
{
    private int x, y;
    public abstract void draw(Graphics g);
}
```



Shape
(abstract)
-x: int
-y: int
+draw(g: Graphics)
+setLocation(x:int, y:int)

- May also have regular methods

Express Yourself

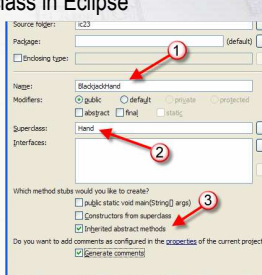
- **Exercise 5:** add an **abstract** method to **Hand**
 - Name the method **evaluateHand()**
 - Returns an **int** representing the value of the specific hand based on the game being played
 - Shoot a screen-shot of the code you added
- **Exercise 6:** add a regular method that **calls** your abstract method in the **Hand** class
 - Name the method **compareTo(Hand other)**
 - Return the **difference** between evaluating the current hand and the other hand. Shoot a screen-shot of the code you added

Extending an Abstract Class

- To **use** the **abstract** class, you **extend** the class and then **override** any and all abstract methods
 - The resulting class is your **concrete** class
- Eclipse will do a lot of the work for you
 - Specify the superclass and then check "Inherited abstract methods"
- Eclipse generates "stubs" for all abstract methods
 - You just "fill in the details"

Express Yourself

- Exercise 7:** create a new class in Eclipse
 - Name: **BlackjackHand**
 - Specify the **Hand** class as the **superclass**
 - Make sure you have inherited abstract methods checked
- Implement method
 - Sum values in hand
 - Face value, 10s, 11

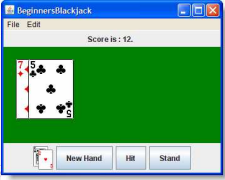


Express Yourself

- Exercise 8:** add some testing code to the bottom of the **BeginnersBlackjack init()** method
 - Call the **newDeck()** method
 - Initialize player by using the **BlackjackHand()** constructor. Note that the variable is a **Hand** but the object it refers to is a **BlackjackHand**
 - Remove the top two cards from the deck and add them to the player's hand.
 - Pass the **player** and the **playerPile** to the **showCards** method that is already written
 - Run and snap a picture

Express Yourself

- **Exercise 9:** write a `showScore()` method that takes a `Hand` object as its parameter
 - Inside the method call the `evaluateHand()` method and display the result in the `statusLbl` on the screen.
 - Call `showScore()` with your `player` variable
 - Shoot a screen-shot.



What Are Interfaces?

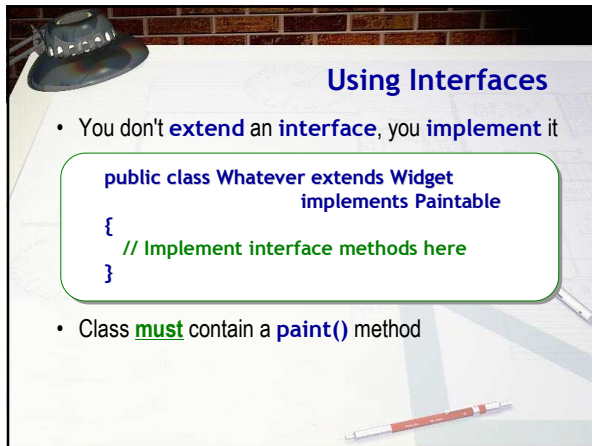
- Something like an **extremely** abstract class
 - Groups together otherwise **unrelated** classes
- An abstract class works with **related** classes:
 - Provides generic behavior using common methods
 - May contain common fields used by all subclasses
 - May contain abstract methods that must be overridden
- An interface specifies what methods **must** be written
 - An interface **can** contain **public static** fields
 - An interface **cannot** contain instance variables
 - An interface can contain **only abstract** methods

Defining Interfaces

- Place in a file with the extension `.java`, just like a class
- Use the keyword **interface** instead of **class**

```
public interface Paintable
{
    public void paint(Graphics g);
}
```

- All methods are **automatically abstract**
- You don't have to use the **abstract** keyword
- The interface and all methods **must** be public

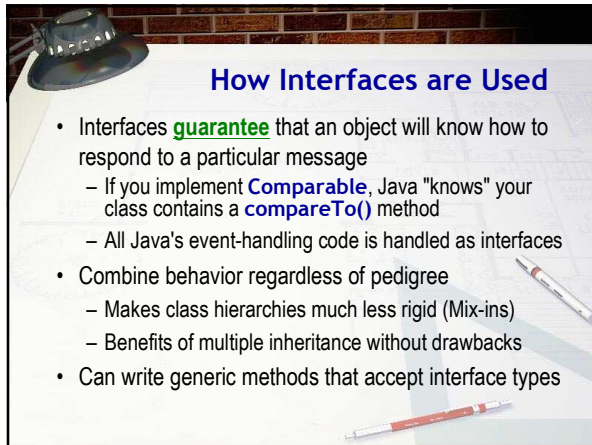


Using Interfaces

- You don't **extend** an **interface**, you **implement** it

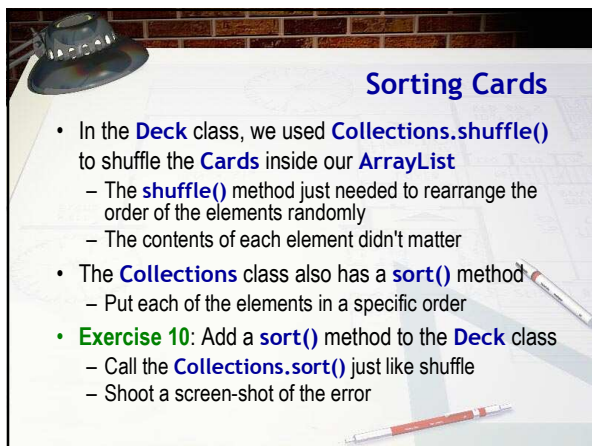
```
public class Whatever extends Widget
    implements Paintable
{
    // Implement interface methods here
}
```

- Class **must** contain a **paint()** method



How Interfaces are Used

- Interfaces **guarantee** that an object will know how to respond to a particular message
 - If you implement **Comparable**, Java "knows" your class contains a **compareTo()** method
 - All Java's event-handling code is handled as interfaces
- Combine behavior regardless of pedigree
 - Makes class hierarchies much less rigid (Mix-ins)
 - Benefits of multiple inheritance without drawbacks
- Can write generic methods that accept interface types



Sorting Cards

- In the **Deck** class, we used **Collections.shuffle()** to shuffle the **Cards** inside our **ArrayList**
 - The **shuffle()** method just needed to rearrange the order of the elements randomly
 - The contents of each element didn't matter
- The **Collections** class also has a **sort()** method
 - Put each of the elements in a specific order
- Exercise 10:** Add a **sort()** method to the **Deck** class
 - Call the **Collections.sort()** just like shuffle
 - Shoot a screen-shot of the error

The Problem with Sorting

- The `sort()` method doesn't care what kind of thing we want to sort, but it doesn't know how to sort **Cards**!
 - The **Card** class itself has to tell it which card must go first and when two cards are equal
 - Do this by implementing the **Comparable** interface
- **Exercise 11:** Add `implements Comparable<Card>` to the **Card** class header.
 - Shoot a screen-shot of the error then Quick Fix
- Implementing an interface is a **promise** that your class **will** provide a particular method or methods.

The Comparable Interface

- To make `sort()` work, we need to add a method with this signature:

```
// a.compareTo(b) returns 0, >0, or <0  
public int compareTo(Card otherCard) { ... }
```

- If two cards are equal, return 0
 - If **a** is **< b** then return **< 0**
 - If **a** is **> b** then return **> 0**
- **Exercise 12:** add `compareTo()` in rank-major order

Implementing compareTo

- Here are some hints for implementing the method
 - If `this == otherCard` then comparing yourself
 - Enumerated types (rank, suit) all have `compareTo`
 - Get the difference between the two ranks and suits
 - Just store the difference in a pair of int variables
 - If the ranks are the same, return the suit difference
- **Exercise 13:** include **CardSortDemo** in the build path, run it, press each button, and shoot 3 screen-shots

Java's Event-handling

- Java uses interfaces for all of its event handling
 - These interfaces are called "listeners"
 - Defined in the `java.awt.event` package
- Events are "external" occurrences that tell you something has "happened" to your program:
 - The user has clicked or moved the mouse
 - User has pressed or released a key on the keyboard
 - The user has entered some text into a text field
 - The user has made a selection from a menu
- **Implement interfaces** to respond to these events

How Event Handling Works

- Any widget can generate (or fire) an event
 - Different components fire different kinds of events
- Any class (not just components) can register to receive notification and respond to an event
 - Objects are called "listeners", "subscribers" or "targets"
 - Components maintain a list of all listener objects that should be notified when an event occurs
- To respond to an event, two things must occur:
 - The **class** must implement the correct interface
 - The **object** must register or subscribe to the listener list maintained by a particular object

Step 1: Preparation

- To make our **Dot** class "mouse aware" we must:
 - Import that package that contains the event interfaces
 - Add **implements MouseMotionListener** to the class header

```
13 import java.awt.*;
14 import java.awt.event.*;
15
16 public class Dot extends Component
17     implements MouseMotionListener
18 {
19     private int radius;
20 }
```

- As soon as we do that, our code stops compiling

Step 2: Subscribe

- To **subscribe**, tell the generator to put the responder on the "subscription list" using a specific method
 - **addMouseListener()** for move/drag events
 - Generator is the receiver, subscriber is the argument
 - For the **Dot** class, put this code in the constructor

```

21:  /**
22:   * Set the default radius, and colors for each Dot
23:   */
24:  public Dot()
25:  {
26:      setRadius(10);
27:      setBackground(Color.green);
28:      setForeground(Color.red);
29:      addMouseListener(this);
30:  }

```

Step 3: Implement

- Each event interface requires different methods
 - Here's the **MouseListener** interface
- Easiest way to add the methods is Eclipse **Quick Fix**
 - Choose "Add unimplemented methods"
 - Now you can **commit and run**

```

12:  /**
13:   * import java.awt.*;
14:   * import java.awt.event.*;
15:   */
16:  public class Dot extends Component
17:  {
18:      // Add unimplemented methods
19:      // Make type 'Dot' abstract
20:      // Rename in file (Ctrl+2, R direct access)

```

Adding Behavior

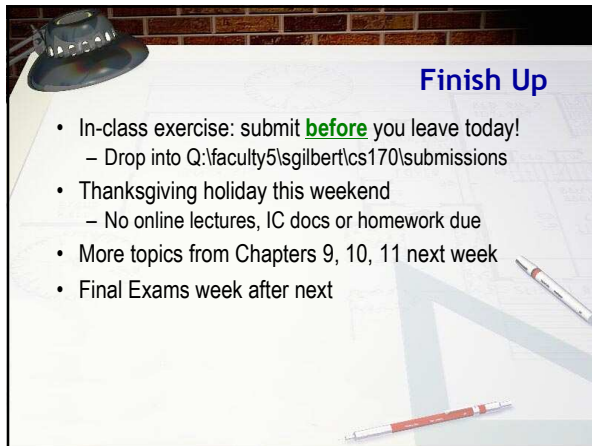
- Nothing happens!** We still need to write code
 - Locate the position of the mouse pointer
 - Convert coordinates into program coordinates
 - Set the location of the dot to the new coordinates

```

51:  /**
52:   * Move the Dot's position
53:   */
54:  public void mouseDragged(MouseEvent e)
55:  {
56:      Point mouseLoc = e.getPoint();
57:      mouseLoc.translate(this.getWidth() / 2, this.getHeight() / 2);
58:      setLocation(mouseLoc);
59:  }
60:  }
61:  }

```

Exercise 14:
Run your applet,
& snap a pic.
Paste your code.



Finish Up

- In-class exercise: submit **before** you leave today!
 - Drop into Q:\faculty5\sgilbert\cs170\submissions
- Thanksgiving holiday this weekend
 - No online lectures, IC docs or homework due
- More topics from Chapters 9, 10, 11 next week
- Final Exams week after next
